

TypeScript fundamentals exercises

Instructions

Complete the exercises in the TypeScript Playground with strict type checking enabled.

Each exercise can be completed independently unless it refers to code from an earlier part of the same exercise. Do not remove a type error by changing the type to `any`.

Section A: Basic types and type decisions

Exercise 1: Catch an incorrect value before the code runs

The following JavaScript function calculates the price of several tickets:

```
function calculateTicketCost(price, quantity) {  
    return price * quantity;  
}  
  
console.log(calculateTicketCost(250, 3));  
console.log(calculateTicketCost(250, "three"));
```

Add TypeScript types to the function so that:

- `price` must be a number.
- `quantity` must be a number.
- The function returns a number.
- The second function call is rejected by TypeScript.

After correcting the code, explain when TypeScript detects the incorrect argument and how this differs from discovering the problem while the program is running.

Exercise 2: Use inference where it is sufficient

TypeScript can infer the types of these variables:

```
const courseTitle = "TypeScript fundamentals";  
let seatsRemaining = 18;  
let registrationOpen = true;  
let registrationStatus = "open";
```

Answer the following questions:

1. What type does TypeScript infer for each variable?
2. Which of these assignments are accepted?

```
seatsRemaining = 12;
seatsRemaining = "12";

registrationOpen = false;
registrationStatus = "closed";
```

1. The registration status must only be "open", "full", or "cancelled". Add the type information needed to enforce this rule.
2. Leave the other variables without explicit annotations unless an annotation changes what values they are allowed to hold.

Exercise 3: Choose a suitable type for each value

A training session stores the following information:

- The session may cover any number of topics.
- The session time always contains a start time followed by an end time.
- The latest assessment result is either a numeric score or "not attempted".

Create suitable declarations for these values:

```
Topics: Functions, Objects, TypeScript
Start time: 10:00
End time: 13:00
Latest result: not attempted
```

Your solution should use:

- An array for the topics
- A tuple for the session time
- A union type for the assessment result

After creating the declarations, update the latest result to 84.

Exercise 4: Restrict the values stored in an array

The following score list contains both numeric scores and a record of an absence:

```
let scores: number[] = [78, 84, "absent", 91];
```

The list must follow these rules:

- A recorded score is a number.
- A missed assessment is recorded as "absent".
- Other strings such as "late" or "missing" must not be accepted.

Correct the type of the array. Then add:

- Another numeric score
- Another "absent" entry
- A commented example showing that "late" is rejected

Exercise 5: Work safely with a union type

A member ID may come from one of two systems:

- Internal IDs are numbers.
- External IDs are strings.

Complete this function:

```
function formatMemberId(id: string | number): string {  
  // Write your code here.  
}
```

Apply these rules:

```
formatMemberId(204);           // "MEM-204"  
formatMemberId("ext-17");      // "EXT-17"
```

Check the value's type before using string-specific or number-specific behaviour. Do not convert the value before deciding which case applies.

Exercise 6: Create a typed package scan log

A package scan stores three values in a fixed order:

1. The tracking code
2. The package weight
3. The scan result

Apply these rules:

- The tracking code is a string.
- The weight is either a number or "unknown".
- The scan result is "accepted", "held", or "rejected".

Create suitable types for the weight, result, and complete scan record.

Then create a scan log containing these records:

Tracking code	Weight	Result
PKG-101	2.4	accepted

Tracking code	Weight	Result
PKG-102	unknown	held
PKG-103	7.8	rejected

The scan log should accept additional valid records but reject a result such as `"delivered"`.

Section B: Objects, interfaces, and reusable shapes

Exercise 7: Describe nested service request data

The following object represents a maintenance request:

```
const request = {
  requestId: "SR-104",
  issue: "Water leakage",
  priority: "high",
  resident: {
    name: "Anita Rao",
    apartment: "B-204",
  },
};
```

Create TypeScript definitions that apply these rules:

- The priority must be `"low"`, `"medium"`, or `"high"`.
- A resident has a name and apartment number.
- A service request has an ID, issue, priority, and resident.
- The resident should use a separate reusable object type.

Apply the completed service request type to the supplied object.

Exercise 8: Use a type alias and an interface for different purposes

A support ticket follows these rules:

- Its status is `"open"`, `"in-progress"`, or `"resolved"`.
- Every ticket has an ID, subject, and status.
- The ID cannot be changed after the ticket is created.
- An assigned agent is optional because new tickets may be unassigned.

Create:

- A type alias for the possible ticket statuses

- An interface for the complete support ticket
- One valid unassigned ticket
- One valid assigned ticket

Briefly explain why the status is represented by a type alias while the ticket itself is represented by an interface.

Exercise 9: Use an optional property safely

A phone number is not required when an attendee registers:

```
interface Attendee {
  name: string;
  phone?: string;
}

const attendee: Attendee = {
  name: "Kabir",
};

console.log(attendee.phone.toUpperCase());
```

Correct the final part of the code so that it prints:

- The phone number in uppercase when it exists
- "Phone number not provided" when it does not exist

Keep `phone` optional. Do not add a placeholder phone number to the object.

Exercise 10: Compare `const` and `readonly`

Read the following code:

```
interface CourseRegistration {
  readonly registrationId: string;
  learnerName: string;
  status: "active" | "completed";
}

const registration: CourseRegistration = {
  registrationId: "REG-100",
  learnerName: "Nina",
  status: "active",
};
```

Consider these changes:

```

registration.learnerName = "Riya";

registration.registrationId = "REG-101";

registration = {
  registrationId: "REG-102",
  learnerName: "Farhan",
  status: "active",
};

```

For each change, state whether TypeScript accepts or rejects it.

Then explain:

1. What `readonly` prevents in this example
2. What `const` prevents
3. Why declaring an object with `const` does not make all its properties readonly

Exercise 11: Extend and combine existing types

A platform already uses this interface:

```

interface UserProfile {
  readonly id: number;
  name: string;
  email: string;
}

```

A mentor profile also needs:

- An array of areas of expertise
- The number of years of experience

Availability is stored separately:

```

type Availability = {
  availableDays: string[];
  acceptsNewLearners: boolean;
};

```

Create a mentor profile by extending `UserProfile`.

Then create a type for an available mentor that combines the mentor profile with `Availability`.

Create one valid available mentor and explain what information comes from each original type.

Exercise 12: Model workshop registration data

A workshop registration contains information about the workshop and the participant.

Apply these requirements:

Workshop

- A readonly numeric ID
- A title
- A delivery mode that can be "online" or "in-person"
- A status that can be "scheduled", "completed", or "cancelled"
- An optional venue

Venue

- A building name
- A room name or number

Participant

- A readonly numeric ID
- A name
- An optional dietary preference

Registration

- A readonly registration code
- A workshop
- A participant
- A registration date

Create suitable TypeScript definitions for this data.

Then create:

1. An online workshop registration without a venue or dietary preference
2. An in-person workshop registration with a venue and dietary preference

Section C: Functions, classes, and generics

Exercise 13: Calculate the total value of an order

An order contains items with a name, unit price, and quantity:

```
interface CartItem {  
  name: string;  
  price: number;
```

```
    quantity: number;
}
```

Write a function named `calculateOrderTotal`.

The function should:

- Accept an array of cart items
- Multiply each item's price by its quantity
- Add the item totals together
- Return a number
- Include explicit parameter and return types

Test it with:

```
const orderItems = [
  { name: "Notebook", price: 120, quantity: 2 },
  { name: "Pen set", price: 80, quantity: 1 },
];
```

The expected result is `320`.

Exercise 14: Type an arrow function with a default parameter

A session contains a title and duration:

```
interface Session {
  title: string;
  durationMinutes: number;
}
```

Write an arrow function named `getLongSessions`.

The function should:

- Accept an array of sessions
- Accept a minimum duration
- Use `60` minutes when no minimum is supplied
- Return the sessions whose duration is equal to or greater than the minimum
- Return a `Session[]`

Test the function with:

```
const sessions = [
  { title: "Type annotations", durationMinutes: 45 },
```



```
{ title: "Classes", durationMinutes: 75 },  
];
```

Call it once without a minimum duration and once with a minimum of 40.

Exercise 15: Handle an optional parameter

Write a function named `formatInvoice` with these parameters:

```
total  
currency  
discountPercent
```

Apply these rules:

- `total` is required and must be a number.
- `currency` defaults to `"INR"`.
- `discountPercent` is optional.
- When no discount is supplied, the original total is used.
- When a discount is supplied, it is deducted before the value is formatted.
- The result must contain two decimal places.

The function should produce these results:

```
formatInvoice(1000);  
// "INR 1000.00"  
  
formatInvoice(1000, "INR", 10);  
// "INR 900.00"  
  
formatInvoice(1000, "USD");  
// "USD 1000.00"
```

Make sure the function does not treat `discountPercent` as a number until its value has been checked.

Exercise 16: Replace repeated objects with a class

The following room objects repeat the same properties and methods:

```
const room101 = {  
  roomNumber: 101,  
  guestName: undefined as string | undefined,  
  
  checkIn(name: string) {  
    this.guestName = name;  
  },  
};
```

```

    checkOut() {
        this.guestName = undefined;
    },
};

const room102 = {
    roomNumber: 102,
    guestName: undefined as string | undefined,

    checkIn(name: string) {
        this.guestName = name;
    },

    checkOut() {
        this.guestName = undefined;
    },
};

```

Replace them with a class named `HotelRoom`.

Each room should have:

- A readonly room number
- An optional guest name
- A constructor that receives the room number
- A method for checking in a guest
- A method for checking out a guest
- A method named `getStatus`

`getStatus` should return `"Available"` when the room has no guest. When the guest name is `"Dev"`, it should return `"Occupied by Dev"`.

Create two room instances and check a guest into only the first room.

After testing the class, identify:

1. The class
2. The two instances
3. The purpose of `new`
4. What `this` refers to inside `checkIn`
5. Why changing the first room does not change the second room

Exercise 17: Control access to an account balance

Create a class named `BankAccount`.

Apply these rules:

- The account number is public and readonly.

- The balance is private.
- The constructor receives an account number and an optional opening balance.
- The opening balance defaults to `0`.
- `deposit` accepts a positive amount and adds it to the balance.
- `withdraw` rejects amounts that are not positive or exceed the current balance.
- A successful withdrawal changes the balance and returns `true`.
- A rejected withdrawal leaves the balance unchanged and returns `false`.
- `getBalance` returns the current balance.

Test the class with:

```
const account = new BankAccount("AC-501", 1000);

account.deposit(500);
account.withdraw(200);
```

The final balance should be `1300`.

Add commented examples showing that outside code cannot:

- Assign a new account number
- Read the private balance directly
- Assign a new balance directly

Exercise 18: Read a protected property through a subclass

Read this code:

```
class StaffMember {
    protected department: string;

    constructor(department: string) {
        this.department = department;
    }
}

class TeamLead extends StaffMember {
    getDepartment(): string {
        return this.department;
    }
}

const lead = new TeamLead("Engineering");
```

Consider these two lines:

```
console.log(lead.getDepartment());  
console.log(lead.department);
```

Answer the following questions:

1. Which line is accepted by TypeScript?
2. Why can `TeamLead` use `department` inside the class?
3. Why can outside code not read `lead.department` directly?
4. How would the result differ if `department` were declared `private`?

Exercise 19: Preserve the item type with a generic function

The following function accepts any array, but the return value loses its original type information:

```
function getFirst(items: any[]) {  
    return items[0];  
}
```

Rewrite the function using a generic type parameter.

The completed function should:

- Accept an array containing values of type `T`
- Return `T | undefined`
- Preserve `string` when called with a string array
- Preserve `number` when called with a number array
- Represent the possibility that the array is empty

Test it with:

```
getFirst(["Types", "Interfaces"]);  
getFirst([78, 84, 91]);  
getFirst<number>([]);
```

Explain what the generic version preserves that the `any` version does not.

Exercise 20: Create a reusable generic result type

Several operations in an application return the same general structure:

- Whether the operation succeeded
- A message
- Optional data produced by the operation

Create a generic interface named `Result<T>` for this structure.

Use this course type:

```
interface CourseSummary {  
  id: number;  
  title: string;  
}
```

Create:

1. A successful course result containing course data
2. A failed course result without course data

Then define a small `LearnerSummary` interface and use the same generic result type to create a successful learner result.

Explain how the value supplied for `T` changes the type of `data` without changing the surrounding result structure.

Final exercise: Build a small course enrolment register

A training centre needs a TypeScript program that registers learners for one course. The program will run in the TypeScript Playground and print its results to the console.

Part 1: Model the data

A course has:

- A readonly numeric ID
- A title
- A delivery mode of `"online"` or `"in-person"`
- A numeric capacity
- An optional room

A learner has:

- A readonly numeric ID
- A name
- An optional email address

An enrolment has:

- A readonly enrolment code
- A course
- A learner
- A status of `"active"` or `"cancelled"`

An operation result contains:

- A boolean named `success`

- A message
- Optional data whose type depends on the operation

Create suitable TypeScript definitions for this data. Use a generic type for the operation result.

Part 2: Describe the course location

Write a function named `getCourseLocation`.

It should return:

- `"Online"` for an online course
- The room name for an in-person course with a room
- `"Room not assigned"` for an in-person course without a room

Part 3: Register learners

Create a class named `EnrolmentRegister`.

The class should contain:

- A public readonly course
- A private array of enrolments
- A constructor that receives the course
- A method named `enrol`
- A method named `getActiveEnrolments`

The `enrol` method receives a learner.

When the course has space, it should:

- Create an active enrolment
- Add it to the private array
- Return a successful result containing the enrolment

When the course is full, it should return an unsuccessful result with this message:

The course is full

`getActiveEnrolments` should return only enrolments whose status is `"active"`.

Part 4: Test the program

Create this course:

```
ID: 301
Title: TypeScript practice lab
Delivery mode: in-person
```

Capacity: 2
Room: Lab 4

Create three learners and try to enrol all three.

Print:

1. The course location
2. The message returned by each enrolment attempt
3. The number of active enrolments

The first two enrolments should succeed. The third should fail because the course capacity is two.